

6교시: Duplicate Request와 Idempotency Gap

Week 3 Day 2 Lesson 6

health check 기본: /health, liveness, readiness 비교

애플리케이션의 상태를 확인하여 안정적인 운영과 트래픽 제어를 가능하게 합니다.

/health 엔드포인트

애플리케이션이 자신의 상태를 외부에 노출하는 표준 HTTP 엔드포인트

외부 클라이언트 (모니터링, 로드밸런서 등) → GET /health → 애플리케이션 /health

응답 예시

✓ **정상(Healthy)** 200 OK

```
{
  "status": "UP",
  "service": "order-service",
  "version": "1.0.0",
  "timestamp": "2025-05-27T10:00:00Z"
}
```

✗ **비정상(Unhealthy)** 503 Service Unavailable

```
{
  "status": "DOWN",
  "service": "order-service",
  "error": "DB connection failed",
  "timestamp": "2025-05-27T10:00:00Z"
}
```

Liveness (생존 확인)

애플리케이션 프로세스가 살아있는지 확인
프로세스가 죽었으면 재시작(Restart)

확인 대상

애플리케이션 프로세스 → 실행 중 (Running) → 생존(Salve) 응답

동작 방식

- 주기적으로 liveness probe 호출 (예: 10초)
- 일정 시간 내 응답 없거나 실패 시 프로세스 재시작
- 외부 의존성(DB, Redis 등)은 보지 않아도 됨

응답 예시

✓ **프로세스 정상(Alive)** 200 OK
응답 본문 예: {"status": "alive"}

✗ **프로세스 비정상(Not Alive)** 503 Service Unavailable
응답 없음 또는 에러 응답

주요 목적

프로세스가 죽으면 자동으로 재시작하여 가용성 유지

Readiness (준비 상태 확인)

애플리케이션이 트래픽을 받을 준비가 되었는지 확인
준비되지 않았으면 트래픽에서 제외

확인 대상

애플리케이션 프로세스 → 의존성 확인 (Dependency) → 준비(Ready) 응답

동작 방식

- 주기적으로 readiness probe 호출 (예: 5초)
- 의존성(DB, Redis, 외부 API 등)까지 확인
- 실패 시 로드밸런서에서 트래픽 제외(Drain)

응답 예시

✓ **준비 완료(Ready)** 200 OK
응답 본문 예: {"status": "ready"}

✗ **준비되지 않음(Not Ready)** 503 Service Unavailable
응답 본문 예: {"status": "not_ready", "reason": "DB down"}

주요 목적

준비되지 않은 인스턴스로 트래픽이 가지 않도록 보호

한눈에 비교	항목	/health 엔드포인트	Liveness (생존 확인)	Readiness (준비 상태 확인)
목적		상태 정보를 외부에 제공	프로세스 생존 확인 및 재시작	트래픽 수신 가능 여부 결정
확인 범위		상황에 따라 다름	프로세스 자체	프로세스 + 외부 의존성
실패 시 조치		모니터링/알림	프로세스 재시작	트래픽 제외(Drain)
대표 응답 코드		200 OK / 503 Service Unavailable	200 OK (alive) / 503 (not alive)	200 OK (ready) / 503 (not ready)
사용 예시		모니터링, 헬스 대시보드	Kubernetes livenessProbe	Kubernetes readinessProbe, 로드밸런서 헬스체크

베스트 프랙티스

- ✓ liveness는 가볍고 빠르게 (외부 의존성 제외)
- ✓ readiness는 필요한 의존성만 확인
- ✓ 의존성 타임아웃 설정으로 빠른 실패 유도
- ✓ 적절한 probe 주기와 임계값 설정
- ✓ 헬스 체크 결과는 구조화된 JSON 응답 권장

수업 목표

- 같은 요청이 두 번 들어왔을 때 중복 업무가 만들어지는 문제를 확인한다.
- request_id와 idempotency key의 차이를 설명한다.
- retry, timeout, 사용자의 더블클릭, 네트워크 재전송이 왜 위험한지 이해한다.
- 중복 방지 설계 후보를 정리한다.

사고 시나리오

현실에서 같은 요청은 여러 이유로 두 번 들어올 수 있다.

원인	예
사용자 더블클릭	주문 버튼을 두 번 누름
client retry	timeout 후 같은 요청 재전송
gateway retry	upstream 응답 지연으로 재시도
mobile network	응답 유실 후 재시도
operator replay	장애 복구 중 같은 event 재처리

Day 1에서는 정상 주문 하나를 만들었다. Day 2에서는 같은 request id로 두 번 보내서 어떤 일이 생기는지 본다.

실행

```
cd week3/day2/labs/incident-scenarios
./04_duplicate_request.sh
```

보야 할 Evidence

Evidence	질문
orders table	같은 request id row가 여러 개인가
audit_logs	order_created, order_processed가 중복되는가
worker log	두 주문 모두 처리됐는가
queue length	중복 event가 남았는가

핵심 개념

```
request id = 추적용
idempotency key = 중복 처리 방지용
```

x-request-id가 같다고 해서 현재 API가 중복을 막아주지는 않는다. 코드가 request id에 unique constraint를 걸거나 idempotency table을 조회하지 않기 때문이다.

해석

관찰	의미
같은 request id의 order row 2개	중복 주문 발생
audit event도 2세트	worker도 각각 처리
HTTP 요청 모두 성공	API 관점에서는 정상
사용자 관점	한 번 주문했다고 생각했는데 두 번 생성될 수 있음

설계 대안

방식	설명
unique idempotency key	같은 key로 두 번째 요청이 오면 기존 결과 반환
request log table	요청 처리 상태를 저장하고 재시도 시 조회
DB unique constraint	업무적으로 중복되면 안 되는 키에 제약
event id deduplication	worker가 이미 처리한 event id를 기록
idempotent update	같은 event를 여러 번 처리해도 결과가 같게 설계

retry와 연결

retry는 필요하지만 위험하다.

retry 상황	위험
client가 timeout 후 재시도	첫 요청이 실제로는 성공했을 수 있음
worker가 error 후 재시도	DB update가 이미 일부 수행됐을 수 있음
queue redelivery	같은 event가 다시 전달될 수 있음
operator가 수동 재발행	이미 처리된 주문을 다시 처리할 수 있음

따라서 retry를 말할 때는 항상 idempotency를 같이 말해야 한다.

운영 리포트 문장

동일 request_id로 주문 생성 요청을 두 번 전송하자 orders table에 두 개의 row가 생성되었다.

audit_logs에도 order_created/order_processed event가 각각 기록되었다.

현재 request_id는 추적에는 사용되지만 중복 방지에는 사용되지 않는다.

중복 주문 방지를 위해 idempotency key 저장소 또는 업무 unique constraint가 필요하다.

Kubernetes 연결

Kubernetes는 pod 재시작과 replica 유지에는 도움을 준다. 하지만 중복 요청과 idempotency 문제는 application 설계 문제다.

문제	Kubernetes 역할
worker pod 죽음	재시작 가능
replica 부족	scale out 가능
같은 event 중복 처리	직접 해결하지 않음
idempotency 보장	application/database 설계 필요

핵심 포인트

MSA에서 안정성을 말할 때 retry만 말하면 반쪽이다.

```
retry without idempotency  
= duplicate risk
```

Evidence Note

```
# W3D2S6 Duplicate Request  
- request_id:  
- order row count:  
- audit row count:  
- duplicate symptom:  
- request_id role:  
- idempotency design:
```